CPS188: Lab 08 - Structures, Binary Files and Personal Libraries
Name: Iqraa Wahid
Student Number: 501287410

Problem #1:

Algorithm:

1. Read the IP addresses and nicknames from a file, and store them in a structure array
2. Check if two addresses belong to the same local network
3. Output the pair of addresses that belong to the same local network
4. Output the list of addresses and their corresponding nicknames

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 300 //maximum 300 addresse
#define NICKNAME_LENGTH 20 //length of nickname

//The following structure will store the nicknames and IP adresses
typedef struct{
     int aa,bb,cc,dd; //components for the integers of the IP
addresses
     char nicknames[NICKNAME_LENGTH];//stores an associated nickname
which is up to 20 characters
}address_t;

//The following function will check whether two IP adresses are on the
same network or not
int localnet(address_t address1, address_t address2) {
    return (address1.aa == address2.aa && address1.bb == address2.bb);
}

int main(void) {
    address_t address[MAX];
    int counter = 0;//counter

     //declaring file
    FILE *myfile;

    //opening file in reading mode
    myfile=fopen("L8_ip.txt", "r");
```

```
    //encountering error opening file
    if (myfile==NULL) {
        printf("There was an error opening the file 'L8_ip.txt'.\n");
        return 1;
    }

    //Reading addresses and nicknames from the file
    while(fscanf(myfile, "%d.%d.%d.%d %s", &address[counter].aa,
&address[counter].bb,
           &address[counter].cc, &address[counter].dd,
address[counter].nicknames)==5) {//when all adresses and nicknames
have been read then continue
        if (address[counter].aa==0 && address[counter].bb==0 &&
address[counter].cc==0 && address[counter].dd==0) {
            break;
        }
        counter++;//update value
        if (counter>=MAX) break;  // This would avoid exceeding the
maximum amount of addresses
    }//closing while loop
    fclose(myfile);//closing file

    //Using 2Dfor loop: Looking for pairs on the same local network
    for (int i=0; i<counter;i++) {
        for (int j=i+1;j<counter;j++) {
            if (localnet(address[i], address[j])) {//if they are on
the same local network:
                printf("Servers %s and %s are on the same local
network.\n", address[i].nicknames, address[j].nicknames);
            }
        }
    }

    //Output:
    printf("\nHere are the full list of addresses and nicknames:\n");
    for (int i=0;i<counter;i++) {
        printf("%d.%d.%d.%d %s\n", address[i].aa, address[i].bb,
address[i].cc, address[i].dd, address[i].nicknames);
    }
    return 0;
```

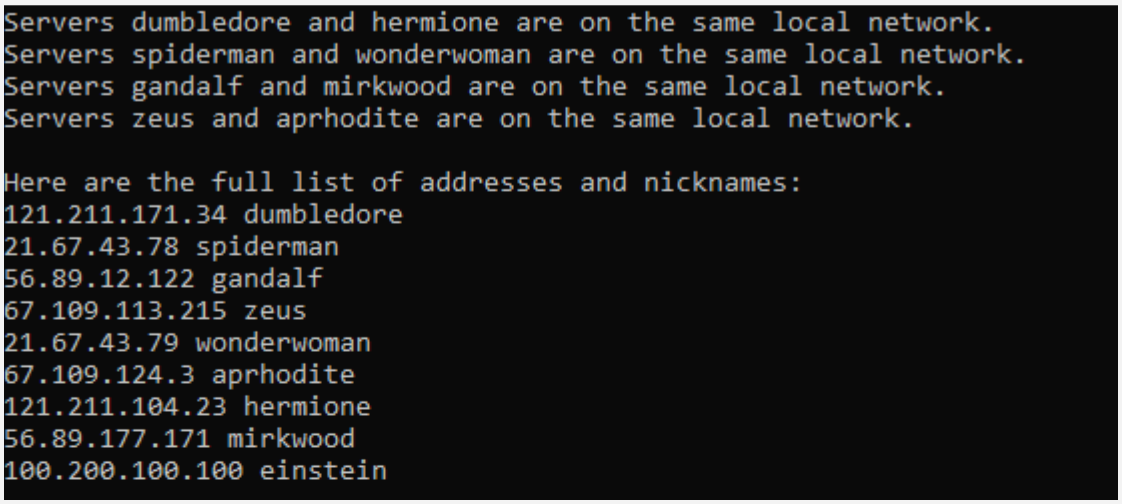
```
}
```

Output:

```
Servers dumbledore and hermione are on the same local network.
Servers spiderman and wonderwoman are on the same local network.
Servers gandalf and mirkwood are on the same local network.
Servers zeus and aprhodite are on the same local network.

Here are the full list of addresses and nicknames:
121.211.171.34 dumbledore
21.67.43.78 spiderman
56.89.12.122 gandalf
67.109.113.215 zeus
21.67.43.79 wonderwoman
67.109.124.3 aprhodite
121.211.104.23 hermione
56.89.177.171 mirkwood
100.200.100.100 einstein
```

Screenshot:



Problem #2:

Algorithm:

1. Read a file and store the numbers from the file into a 10x10 2D array
2. Calculate the following corresponding to the numbers stored in the array: Sum of the main diagonal, the sum of all elements, the average of the last column, the sum of the four corners, and the largest value in the antidiagonal
3. Write the results to into a binary file, and read it back from the binary file
4. Output the results calculated for the array

Code:

```
#include <stdio.h>
#include <stdlib.h>

//Functions with purposes written:
double mainDiagonalSum(double array[10][10]) {//this will caclulate
the sum of the main diagonal
    double sum=0.0;//stores the sum of the main diagonal
    for (int i=0;i<10;i++) {
        sum += array[i][i];
    }
    return sum;//returns the sum obtained
}

double allNumbersSum(double array[10][10]) {//this will caclulate the
sum of all numbers
    double sum=0.0;//stores the sum of all numbers
    for (int i=0;i<10;i++) {
        for (int j=0;j<10;j++) {
            sum += array[i][j];
        }
    }
    return sum;//returns the sum obtained
}

double lastColumnAverage(double array[10][10]) {//this will calculate
the average of the last column
    double sum=0.0;
    double average;
    for (int i=0;i<10;i++) {
        sum += array[i][9];
    }
    average=sum/10.0;
    return average;//returns the average obtained
}

double fourCornersSum(double array[10][10]) {//this will caculate the
sum of the four corners
    double sum = array[0][0] + array[0][9] + array[9][0] +
array[9][9];
    return sum;//returns the sum obtained
```

```
}

double largestAntidiagonal(double array[10][10]) {//this will
calculate the largest antidiagonal
    double maximum = array[0][9];//let maximum be array at elements 0,
1 initially
    for (int i=1;i<10;i++) {
        if (array[i][9-i]>maximum) {//checks if the next value is
greater than the one already set
            maximum = array[i][9 - i];//if the next value is greater,
then that is set as the max
        }
    }
    return maximum;//returns the maximum value
}

int main(void) {
    double numbers[10][10];//declaring a 10 by 10 array to store the
numbers from a file
    FILE *inputFile;//declaring file to read
    FILE *outputFile;//declaring file to write

    //open file in reading mode
    inputFile = fopen("L8_real.txt", "r");

    //if file doesn't open:
    if (inputFile==NULL) {
        printf("There was an error opening the file
'L8_real.txt'!\n");
        return 1;
    }
    //reading the numbers from the file into a 2D array
    for (int i=0;i<10;i++) {
        for (int j=0;j<10;j++) {
            if (fscanf(inputFile, "%lf", &numbers[i][j])!=1) {//If
this isn't true:
                printf("There was an error encountered while reading
the file!\n");
                fclose(inputFile);//closing file
                return 1;
            }
```

```
        }
    }
    fclose(inputFile);//close file

    //calling functions and storing value calculated and obtained
through functions in an array:
    double result[5];//storing the results in an array
    result[0] = mainDiagonalSum(numbers);
    result[1] = allNumbersSum(numbers);
    result[2] = lastColumnAverage(numbers);
    result[3] = fourCornersSum(numbers);
    result[4] = largestAntidiagonal(numbers);

    //writing the results to binary file
    outputFile = fopen("results.bin", "wb");//opening file in writing
binary mode

    //if file isnt created:
    if (outputFile==NULL) {
        printf("There was an error creating this output file for
writing!\n");
        return 1;
    }

    fwrite(result, sizeof(double), 5, outputFile);
    fclose(outputFile);//closing the file

    double readingResults[5];//declaring array
    outputFile = fopen("results.bin", "rb");//opening file in reading
mode

    //if file doesn't open
    if (outputFile==NULL) {
        printf("There was an error opening this output file for
reading!\n");
        return 1;
    }

    fread(readingResults, sizeof(double), 5, outputFile);
    fclose(outputFile);//closing file
```

```
    //Output:
    printf("The sum of the numbers in the main diagonal is %.2f.\n",
readingResults[0]);
    printf("The sum of all the numbers in the array is %.2f.\n",
readingResults[1]);
    printf("The average of the last column is %.2f.\n",
readingResults[2]);
    printf("The sum of the four corners is %.2f.\n",
readingResults[3]);
    printf("The largest number in the antidiagonal is %.2f.\n",
readingResults[4]);

    return 0;
}
```

Output:

```
The sum of the numbers in the main diagonal is 7038.70.
The sum of all the numbers in the array is 54410.40.
The average of the last column is 511.25.
The sum of the four corners is 3478.90.
The largest number in the antidiagonal is 980.80.
```

Screenshot: